

A Formal Semantics of C with OpenMP Parallelism

Ke Du¹[0009–0008–2465–1082], Anshu Sharma², Liyi Li³[0000–0001–8184–0244], and William Mansky¹

¹ University of Illinois Chicago

² The College of William and Mary

³ Iowa State University

Abstract. OpenMP is a popular parallelization framework that lets users transform sequential code into parallel code with a few simple annotations. Unfortunately, it is also easy to inadvertently introduce errors by adding OpenMP pragmas into otherwise correct programs, including both logic errors and race conditions. We present a formal semantics for C code with OpenMP directives, building on the C semantics of the CompCert verified compiler and its extension to concurrency. Our semantics captures subtle interactions between OpenMP directives and variable state that have been obscured by previous OpenMP semantics, and provides a basis for detecting undesired behaviors introduced by incorrect annotations: in particular, any successful execution is guaranteed to be free of data races.

Keywords: OpenMP · Formal Semantics · Concurrency.

1 Introduction

The OpenMP framework [5] provides a straightforward entry point for shared-memory parallel programming. Instead of explicitly creating and managing threads and locks, a programmer can write a sequential program and then use the OpenMP API to instruct the compiler to automatically parallelize the code, e.g., by distributing the iterations of a loop across multiple threads. The core of the API is a set of compiler directives (`#pragmas` in C and C++) that can be added before the code segments to be parallelized, supported by common compilers including GCC and Clang. However, the simplicity of this interface can be deceptive—OpenMP still suffers from all the complexity of shared-memory concurrency, and choosing the wrong directives can easily introduce race conditions or logic errors.

Prior work [3] defined formal semantics for OpenMP for the purpose of race detection, modeling the synchronization effects of key directives and the memory accesses they produce. This is useful for detecting concurrency bugs, but does not fully address the question “what is this OpenMP program allowed to do?” In this work, we aim to answer that question with a formal semantics for OpenMP at the language level, specifically for C programs with OpenMP annotations.

Our semantics builds on CompCert’s C semantics [10] and its extension to concurrency [6], giving us high confidence in our base language semantics; our task is then to accurately capture the allowed behavior of OpenMP directives.

There are three main challenges to giving formal semantics for OpenMP at the language level:

1. The OpenMP API is not a library of functions but a set of compiler directives that modify the behavior of the code they apply to. These directives cannot be modeled as special function calls executed in a single step, as is standard in concurrent extensions of CompCert, since, at a minimum, they may have effects at both the beginning and the end of the affected block.
2. Unlike standard fork-join concurrency, in which an unstructured collection of threads interacts peer-to-peer via shared memory or message passing, OpenMP describes the behavior of dynamically created *teams* of threads, and the validity and effects of directives depend on the team to which the executing thread currently belongs. Any formal model of OpenMP must track the team structure of the executing threads in addition to more usual forms of state.
3. OpenMP directives have subtle effects on local variables: depending on the position of its declaration and the details of the directive, a variable may be treated as shared memory across multiple threads or split into private copies that may be reconciled at the end of a parallel section—and mistakes in variable-handling directives are a major source of OpenMP-induced bugs.

By carefully modeling the scope of OpenMP directives and their effects on team structure and local variables, in addition to their concurrency and memory behavior, our semantics provide a basis for determining the allowed behavior of an OpenMP-annotated program, and thus ultimately for determining whether a given parallelization preserves the intended behavior of the code. Our semantics is stuck in the presence of memory errors or data races, making it useful for bug-finding. The semantics can also serve as a basis for analysis tools that prove the safety and functional correctness of OpenMP C programs.

1.1 A Motivating Example

We illustrate the uses and challenges of our semantics with a typical OpenMP program (shown on the left of Figure 1) and a typical OpenMP mistake (shown on the right). The core mechanism of OpenMP is a set of *directives*, compiler annotations to parallelize a sequential program, implemented as `#pragmas` in C. Some directives are associated with the following code block, binding to them to form a *construct*. Other directives, such as `barrier`, do not associate with a statement and form a construct by themselves.

In the left program, `parallel num_threads(2)` is an OpenMP directive that specifies that a *team* of two threads should execute the associated code in parallel, with all threads executing the same code block. The `omp for` directive on line 5 is always associated with a for-loop, and divides the workload of the associated for-loop among the threads of the executing team. When this program executes, it

<pre> 1 <i>// program starts with one thread t1</i> 2 #pragma omp parallel num_threads(2) 3 <i>{// a team of 2 threads {t1, t2} is created</i> 4 f(); 5 #pragma omp for 6 for (i=0; i<6; i+=1) { 7 <i>// each thread only works a loop partition</i> 8 loop_body 9 } 10 } <i>// end of parallel region, only thread is t1</i></pre>	<pre> 1 int k = 0, s = 0; 2 #pragma omp parallel for 3 reduction(+ : k, s) 4 for (int i = 0; i < 128; i++) { 5 k = k + 3; 6 s = s + k; 7 }</pre>
---	---

Fig. 1: Left: a typical OpenMP program. Right: an incorrect OpenMP program.

begins with a single thread of execution; at line 2, the `parallel` directive creates a team with two threads; and the iterations of the `for`-loop on line 6 are partitioned across the threads, e.g., each thread may execute three steps of the loop.

This illustrates several challenges in defining the semantics of OpenMP:

1. The `for` directive is team-sensitive, i.e., the partitioning of the loop at line 5 depends on the number of threads set up in line 2.
2. OpenMP does not check for race-freedom: if `loop_body` is not safe to run in parallel, the program will not execute correctly.
3. At the end of the `parallel` block at line 10, there is an implicit barrier: every thread in the team must wait until all threads finish their tasks.

Our semantics must track the team structure throughout the program, detect race conditions among threads, and capture the effects of both the start and end of a construct.

Another difficulty arises from the complex interaction between OpenMP directives and local variables, as shown in the right program in Figure 1 (adapted from Ahmed et al. [1]). When run sequentially, this program sums the multiples of 3 from 3 to $3 * 128$. The `reduction` clause aims to divide the work across threads; each thread receives its own copies of `k` and `s`, and the copies are summed at the end of the loop. For `k`, this will indeed produce the right result ($3 * 128 = 384$). However, depending on the distribution of loop iterations among threads, some per-thread values of `k` will be reused in computing `s`, while other values will never be reached, producing an incorrect final value of `s`. This program does not contain a data race—it is not buggy in and of itself—but it does not have the same semantics as the corresponding sequential program, and we can only determine this by modeling the values of `k` and `s` in each thread. Notably, the main prior work on the semantics of OpenMP [3] works at the level of memory operations, obscuring the effects of OpenMP directives on variables and control flow. In contrast, our OpenMP semantics directly models the effects of directives on the C program state, allowing us to detect errors like that on the right of Figure 1.

1.2 Contributions and Roadmap

The main contribution of this paper is, to our knowledge, the first formalization of OpenMP semantics in C at the language level. We formalize 5 constructs and 3 clauses representing commonly used OpenMP directives. In particular, we give the first formal semantics for the `for` and `single` clauses, which control work-sharing within teams, and the `private` and `reduction` clauses, which control the interaction between threads and local variables. Our novel *team tree* construction captures the essence of OpenMP’s execution model and could be reused to define OpenMP semantics in other languages, such as C++ or FORTRAN. Our semantics is based on per-thread permissions to memory, and we prove that any execution in our semantics is data-race-free.

The paper is structured as follows. In Sections 3.1 and 3.2, we describe the syntax of ClightOMP and the team tree model that plays a key role in ClightOMP program states. In Section 3.3, we formally present our operational semantic rules for OpenMP and describe in detail the semantics of the major constructs and clauses of OpenMP: parallel and worksharing constructs, barriers, and privatization and reduction clauses. The ClightOMP semantics is formalized in the Rocq proof assistant. Section 4 describes the properties of the semantics, including a proof of data-race freedom for any execution. Finally, in section 5 we compare with related work, and in section 6 we summarize our results and discuss future work.

2 Background: The Concurrent Permission Machine

Since our goal is to give precise semantics for C programs with OpenMP directives, we begin with the authoritative formal semantics of ordinary C, the Clight semantics of the CompCert verified compiler [10]. However, CompCert’s model of C does not include concurrency—it is strictly a sequential semantics. Several more recent works, including CASCompCert [8] and the Concurrent Permission Machine (CPM) [6], lift Clight to concurrency by defining a multithreaded machine where each step is either a sequential Clight step or a concurrency operation; our semantics builds on the CPM approach. In this section, we explain the parts of the CPM’s program states and step rules that will be necessary to understand our semantics.

A CPM program state has the form $\langle \mathcal{U}, \mathcal{P}, m \rangle$, consisting of a schedule $\mathcal{U}$, a thread pool $\mathcal{P}$, and a shared memory m . The schedule $\mathcal{U}$ is a list of thread IDs that determines the order in which threads interleave; the possible behaviors of a program are its behaviors under all schedules. The thread pool $\mathcal{P}$ maps each thread ID i to its local state $P(i) = \langle \sigma_i, \pi_i \rangle$, where $\sigma_i = \langle s, le, k \rangle$ is a Clight state containing the statement s to be executed, the local variable environment le that maps variable names to their addresses in the memory m , and the continuation k ; π_i is the *permission map* of thread i , describing the thread’s access level to each memory location (None, Nonempty, Readable, Writable, or Freeable). These access levels allow the CPM to detect data races: at each step of execution, the permissions held by all threads must be compatible with each other, so

that e.g. if one thread holds **Writable** access to a location, no other thread can hold **Readable** access. Finally, m is a CompCert memory [4]. For this paper, it can be viewed as a pair $\langle m_v, \pi_m \rangle$ where m_v maps addresses to values, and π_m is a placeholder for the executing thread's permission map. We define a few shorthands: $m(l) \triangleq m.1(l)$ for loading a value from address l , $\text{perm}(m) \triangleq m.2$ for getting the permission map, $m|_\pi \triangleq \langle m.1, \pi \rangle$ for replacing the permission in m with π .

The operational semantics of the CPM includes two kinds of steps. First, an individual thread may take a step according to the sequential Clight semantics $\rightarrow_{sq}$:

$$\text{STEP-CLIGHT} \quad \frac{\mathcal{P}(i) = \langle \sigma_i, \pi_i \rangle \quad \langle \sigma_i, m|_{\pi_i} \rangle \rightarrow_{sq} \langle \sigma'_i, m' \rangle \quad \mathcal{P}[i ::= \langle \sigma'_i, \text{perm}(m') \rangle] = \mathcal{P}'}{\langle i \cdot \mathcal{U}, \mathcal{P}, m \rangle \rightarrow \langle \mathcal{U}, \mathcal{P}', m' \rangle}$$

If i is the next thread scheduled to execute, and its current state is $\langle \sigma_i, \pi_i \rangle$ where σ_i can execute a sequential C statement, then it executes as follows. First, it combines its permissions π_i with the shared memory m to obtain a thread-local “view” of memory $m|_{\pi_i}$, which has all the same values as m but allows only operations that the thread has permission for (e.g., if π_i only has **Readable** access to a location ℓ , then a write to ℓ in $m|_{\pi_i}$ will fail). The local state σ_i and local memory $m|_{\pi_i}$ combine to make an ordinary Clight state, which may step according to Clight's sequential semantics to a new state $\langle \sigma'_i, m' \rangle$. The new memory m' then becomes the new shared memory, and the thread pool records the new local state σ'_i and permissions $\text{perm}(m')$ for thread i .

The STEP-CLIGHT rule applies when the next operation in the scheduled thread is a sequential statement. When it is a concurrency operation, we instead look for a specific rule for that operation in the CPM. For instance, the rule for thread creation is:

$$\text{STEP-SPAWN} \quad \frac{j \notin \mathcal{P} \quad \pi_i = \pi'_i \oplus \pi'_j}{\langle i \cdot \mathcal{U}, \mathcal{P} \left[i ::= \langle \langle \text{spawn}(f, a), le, k \rangle, \pi_i \rangle \right], m \rangle \rightarrow \langle \mathcal{U}, \mathcal{P} \left[i ::= \langle \langle \text{SSkip}, le, k \rangle, \pi'_i \rangle, j ::= \langle \langle f(a), \emptyset, \text{KStop} \rangle, \pi'_j \rangle \right], m \rangle}$$

The in-place update notation $\mathcal{P}[i ::= (\sigma, \pi)]$ is the thread pool $\mathcal{P}$ but thread i has state (σ, π) . If thread i wants to spawn a new thread executing function f with argument a , it can do so by creating a new thread whose code is $f(a)$ with a fresh index j not yet in $\mathcal{P}$. The parent thread's statement is replaced by the no-op statement **SSkip**, so its next step will be decided by its continuation k . The new thread has an empty local variable environment $\emptyset$ and a continuation **KStop**, meaning that the thread halts after $f(a)$. The permissions π_i of i must be split into two parts, one retained by the parent thread (π'_i) and one given to the child thread (π'_j).

The CPM includes rules for thread creation and lock operations (allocation, deallocation, acquire, and release). Taken together, these implement a concurrent

extension of Clight with an SC-DRF (Sequential Consistency for Data-Race-Free programs) memory model: programs with data races are stuck (due to permission conflicts), while programs without data races have sequentially consistent behavior, interleaving the steps of individual threads. We develop our OpenMP semantics by extending the CPM with rules for OpenMP directives like parallel regions and work-sharing constructs, extending the state model as necessary. OpenMP’s memory model is also SC-DRF as long as programs do not use weak-memory `atomic` constructs (which we do not support), so the CPM provides a reasonable basis for capturing the concurrency behavior of C programs with OpenMP directives.

3 Syntax and Semantics of ClightOMP

ClightOMP extends the syntax of Clight [10], the subset of C formalized in CompCert, with OpenMP directives as new statements. The full syntax is defined in Figure 9. In this section, we introduce the syntax and give an operational semantics to these statements, building on Clight and the CPM semantics.

3.1 Syntax: Extending Clight

The extended statements for constructs, such as the one for the parallel construct, $\text{SPar}_{idx} \text{ } nc \text{ } pc \text{ } \vec{rcs} \text{ } s$, have 4 parts in general: **SPar** specifies the kind of directive; *nc*, *pc* and $\vec{rcs}$ are construct-specific *clauses* that specify additional behaviors; *s* is the statement affected by the directive; lastly, the subscript *idx* is an index that uniquely identifies the construct. Concretely, line 2-10 in Figure 1 is encoded as $\text{SPar}_{idx} \text{ } 2 \text{ } [] \text{ } s$, where *idx* is some unique index and *s* is the statement for line 3-10. We explain them in detail.

In addition to **SPar** for the `parallel` directive, we define 4 core statements for directives in total: **SFor** for the `for` directive, **SSingle** for the `single` directive, and **SBarrier** for the `barrier` directive.

The `parallel`, `for`, and `single` directives are lexically associated with the following statement (or block) and semantically affect that statement, and we encode the affected statement as the last argument *s* of **SPar**, **SFor** and **SSingle**. These directives and their associated statements form a *construct*. The `barrier` directive does not associate with a statement and forms a barrier construct on its own.

A construct statement can have 0 or more clauses. The *nc* clause specifies the number of threads in the `parallel` construct⁴. A privatization clause *pc* gives a list of variable names to privatize. Constructs that support reduction have a list of reduction clauses $\vec{rcs}$, and a reduction clause *rc* is a pair of a reduction identifier *rid* that specifies the operation for combining the reduction contributions, and a list of variables to be reduced—for example, at the end of a for

⁴ In the OpenMP standard, the actual number of threads is determined by a number of factors at runtime; currently, we assume that the number is determined statically, but our semantics could be modified to follow the standard more closely.

loop that computes a sum in variable `sum`, we may sum every thread’s contribution to compute the final result using a reduction clause `(+:sum)`. Both `SPar` and `SFor` support a privatization clause and a list of reduction clauses; `SSingle` supports privatization but not reduction. When directives with these clauses are executed, we generate runtime-only instructions `SPriv`, `SPrivEnd`, `SRed` that carry a mix of statically and dynamically recorded information about how to perform privatization and reduction; we discuss these further in Section 3.5.

There are several places in the semantics of OpenMP where we need to know that statements executed by different threads are not merely equal but in fact the same syntactic occurrence of the statement: for instance, threads in a team synchronize at a barrier only if they have all reached the same barrier. For this purpose, we give every construct statement (`SPar`, `SFor`, `SSingle`, `SBarrier`) a unique integer index n . We also generate `SBarriers` at runtime to make implicit barriers explicit, and they need to be further distinguished with an optional statement s and a boolean b ; we discuss these details in Section 3.4.

3.2 Synchronization with the Team Tree

As described in Section 2, the CPM models the execution of concurrent programs by tracking a collection of per-thread states and a single shared memory. For OpenMP, we need to track an additional kind of information: the hierarchical structure of spawned threads, which are organized into *teams*, and the states of their in-progress OpenMP operations. Since the team hierarchy in the program forms a tree (each team is spawned by a single parent thread, which may itself be part of a team), we organize OpenMP-specific state information into a tree that we call the *team tree*, denoted by $\mathcal{T}$, which we include in program states alongside the thread pool $\mathcal{P}$ and memory m . The execution of sequential C code (rule STEP-CLIGHT) is not affected by the addition of the team tree, as regular per-thread execution is independent of OpenMP states.

We define the team tree by defining its *Nodes*. Each node corresponds to a thread of execution. When a thread spawns a team, its node adds child nodes for the team’s threads, along with some *team contexts* that record OpenMP state common to the entire team. Formally:

Definition 1 (Node and Team) A *Node* of a team tree is a tuple (t, tm) where t is a thread ID and tm is either some *Team* or $\perp$. A *Team* is a tuple $(leader, pctx, ectx, mates)$ where *leader* stores the thread ID for the leader of the team (which is always the thread that spawns the team), *pctx* is the parallel context of the team, *ectx* is a team-executed context representing the currently active team-executed construct⁵ or $\perp$, and *mates* is a list of *Nodes* for the members of the team.

⁵ One team-executed context suffices because all threads in a team can be in at most one team-executed construct at a time: the OpenMP standard effectively requires that, inside a parallel region, team-executed contexts cannot be nested. From the standard: “a team-executed region (`for`, `single`, `barrier`) may not be closely nested (nested in the same parallel region) inside a partitioned worksharing region (`for` or

The program starts with the initial team tree $(t_1, \perp)$, containing a single *Node* for the initial thread t_1 and no team.

The parallel context ParCtx_{idx} of a *Team* records the index idx of the parallel construct that spawned it; $ectx$ exists if some member of the team is inside a team-executed construct, and records information specific to that construct. It has the form $\text{ForCtx}_{idx} w$ for a **for** construct, or $\text{SinCtx}_{idx} j$ for a **single** construct. We will explain the definition and the usage of the contexts when we introduce the semantics in the following sections.

The thread that spawns a team becomes the *leader* of this team; it participates in this team as a new node with the same thread ID (therefore this new node is also a child of the original node). Spawned threads can also spawn teams themselves, resulting in nested teams.⁶ Only the leaf node of i in a team tree $\mathcal{T}$ is immediately relevant to its execution, and we call this node the *active* node of i , denoted as $\mathcal{T}(i)$. $\mathcal{T}(i)$ is only defined if the active node for i is unique.

In this section, we will describe the operations on team trees informally; formal definitions are in Figure 12.

3.3 Overview of the Semantics

Our semantic rules for the OpenMP constructs are shown in Figures 2 and 6. As described in the previous section, our step relation is on configurations of the form $\langle \mathcal{U}, \mathcal{P}, \mathcal{T}, m \rangle$, where $\mathcal{U}$ is a thread schedule, $\mathcal{P}$ is a collection of per-thread states including control state (for example, the current *Clight* statement and continuation), local variables, and permissions, $\mathcal{T}$ is a team tree, and m is the shared memory. Each rule begins with a schedule of the form $i \cdot \mathcal{U}$, indicating that i is the next thread to execute, and then explains the effect on the configuration if i is executing one of the OpenMP directives (regular program steps do not affect the team tree, and so use the **STEP-CLIGHT** rule from Section 2 as is). In the rest of this section, we describe each of these rules in detail.

Judgement and Notation Although OpenMP constructs are essentially concurrent, some can be described as local transitions of a single thread, together with synchronizations via the team tree $\mathcal{T}$. This is expressed with the judgement $s \parallel le \parallel m \xrightarrow[h]{i, \mathcal{P}, \mathcal{T}} s' \parallel le' \parallel m'$: it says a thread i updates its statement s and variable environment le (along with the global memory m) to s', le' (and m'), while updating the team tree with h , where h may bind the original thread pool $\mathcal{P}$ and team tree $\mathcal{T}$, and the new team tree is $h(\mathcal{T}, i)$. We omit h if the team

single). We implement the **barrier** construct as a single atomic operation, and all of our team-executed constructs have an implicit barrier at the end of the construct, so no thread can enter a new team-executed construct while some of its teammates are still executing the old one.

⁶ In this work, we only model programs that are executed by one *device*, and all synchronization is due to the OpenMP pragmas that we define. We do not model the **teams** construct, or the notion of a **league** of teams, and all threads are descendants of the initial thread.

$$\begin{array}{c}
\text{SBRB}_{idx,s,r} \triangleq \text{let } s_r := \text{if } r.1 = [] \text{ then SSkip else SBarrier}_{idx,false,s}; \text{SRed } r \text{ in} \\
s_r; \text{SBarrier}_{idx,true,s} \\
\\
\text{STEP-PARALLEL} \\
\frac{|\vec{t}| = nc - 1 \quad r \triangleq (\vec{rc}, le|_{\vec{x}}) \quad \forall j \in \vec{t}. j \notin \mathcal{P} \quad s' \triangleq \text{SPriv } \vec{x} (s; \text{SBRB}_{idx,\text{SPar},r}) \quad \pi_i = \pi'_i \oplus (\bigoplus_{j \in \vec{t}} \pi'_j)}{\langle i \cdot \mathcal{U}, \mathcal{P} [i ::= \langle \langle \text{SPar}_{idx} \ nc \ \vec{x} \ \vec{rc} \ s, le, k \rangle, \pi_i \rangle], \mathcal{T}, m \rangle \rightarrow} \\
\langle \mathcal{U}, \mathcal{P} [i ::= \langle \langle s', le, k \rangle, \pi'_i \rangle, \forall j \in \vec{t}. j ::= \langle \langle s', le, \text{KStop} \rangle, \pi'_j \rangle], \text{spawn_team}(\text{ParCtx}_{idx}, \vec{t})(\mathcal{T}, i), m \rangle \\
\\
\text{STEP-BARRIER} \\
\frac{\vec{t} \triangleq \text{team_tids}(\mathcal{T}, i) \quad \mathcal{T}(i). \text{ctx} \neq \perp \iff s = \text{SFor} \vee s = \text{SSingle} \quad \bigoplus_{j \in \vec{t}} \pi_j = \bigoplus_{j \in \vec{t}} \pi'_j}{\langle i \cdot \mathcal{U}, \mathcal{P} [\forall j \in \vec{t}. j ::= \langle \langle \text{SBarrier}_{idx,b,s}, le_j, k_j \rangle, \pi_j \rangle], \mathcal{T}, m \rangle \rightarrow} \\
\langle \mathcal{U}, \mathcal{P} [\forall j \in \vec{t}. j ::= \langle \langle \text{SSkip}, le_j, k_j \rangle, \pi'_j \rangle], \text{if } b \text{ then } \text{end_ctx}(s)(\mathcal{T}, i) \text{ else } \mathcal{T}, m \rangle \\
\\
\text{STEP-THREAD} \\
\frac{s \parallel le \parallel m|_{\pi_i} \xrightarrow{i, \mathcal{P}, \mathcal{T}}^h s' \parallel le' \parallel m'}{\langle i \cdot \mathcal{U}, \mathcal{P} [i ::= \langle \langle s, le, k \rangle, \pi_i \rangle], \mathcal{T}, m \rangle \rightarrow \langle \mathcal{U}, \mathcal{P} [i ::= \langle \langle s', le', k \rangle, \text{perm}(m') \rangle], h(\mathcal{T}, i), m' \rangle} \\
\\
\text{STEP-FOR} \\
\frac{s' \triangleq \text{set_stmt_to_partition}(s, w(i)) \quad w \triangleq \text{partition}(s) \quad r \triangleq (\vec{rc}, le) \quad s'' \triangleq \text{SPriv } \vec{x} (s'; \text{SBRB}_{idx,\text{SFor},r})}{\text{SFor}_{idx} \ \vec{x} \ \vec{rc} \ s \parallel le \parallel m \xrightarrow{i}^{\text{sync_ctx}(\text{ForCtx}_{idx} \ w)} s' \parallel le \parallel m} \\
\\
\text{STEP-SINGLE} \\
\frac{s' \triangleq \text{SPriv} (\text{if } i = j \text{ then } \vec{x} \text{ else } []) ((\text{if } i = j \text{ then } s \text{ else SSkip}); \text{SBarrier}_{idx,true,\text{SSingle}})}{\text{SSingle}_{idx} \ \vec{x} \ s \parallel le \parallel m \xrightarrow{i}^{\text{sync_ctx}(\text{SinCtx}_{idx} \ j)} s' \parallel le \parallel m}
\end{array}$$

Fig. 2: Semantic rules: parallel, for, single, barrier

tree is not updated. The STEP-THREAD rule in Figure 2 lifts a single-thread step by thread i to the full configuration, writing back the updated s' and le' to the thread pool $\mathcal{P}$, updates $\mathcal{T}$ to $h(\mathcal{T}, i)$, and m to m' . Thread i 's continuation k stays the same. Like STEP-CLIGHT, thread i can only step with $m|_{\pi_i}$, i.e. m equipped with thread i 's initial permission π_i ; we then save $\text{perm}(m')$, the permission in the new memory m' , as thread i 's new permission. STEP-PARALLEL and STEP-BARRIER change the state of multiple threads and are not expressed in this style. We abuse the $le|_{\vec{x}}$ notation to mean the map le with the domain restricted to $\vec{x}$.

We explain the basic semantics of the constructs that enable parallelism and worksharing in Section 3.4. Privatization and reduction clauses can be attached to these constructs to fine-tune their behavior; their semantics are explained in Section 3.5.

3.4 Parallelism and Work-Sharing

```

1 #pragma omp parallel num_threads(5)
2 {
3   #pragma omp for
4     for(int i = 0; i < 100; i++) {
5       printf("%d\n", i);
6     }
7 }

```

Fig. 3: An OpenMP program with `parallel` and `for` constructs.

```

1 #pragma omp parallel num_threads(2)
2 {
3   #pragma omp single
4     { printf("Before\n"); }
5     printf("After\n");
6 }
7

```

Fig. 4: An OpenMP program with a `single` construct.

The Parallel Construct The main structuring construct in OpenMP is the `parallel` construct, which creates a team of threads to execute a parallel region. When a thread i reaches a `SPar` statement, the rule STEP-PARALLEL forks a team of threads executing the region’s body s in parallel. For example, in Figure 3, a team of 5 threads executes lines 2–6 in parallel. Thread i also participates in the team and becomes the *primary thread* of the team, so we only need to add $nc - 1$ new threads. We add these threads both to the team tree $\mathcal{T}$ via the `spawn_team` operation and to the thread pool $\mathcal{P}$, where we record each thread’s code, local environment, and permissions. At a first pass, the code executed by the spawned threads is $(s; \text{SBarrier}_{\text{true}, \text{SPar}})$: each thread executes the body s of the parallel region, and then a barrier synchronizes all the threads in the team and ends the parallel region.

For the permissions, all teammates split thread i ’s original permissions arbitrarily—a given memory location may be written by a single thread in the team or read by all of them (but not both). In this case, the original permission π_i is redistributed among the team $(\pi'_i \text{ and } \pi'_j \text{ for } j \in \vec{t})$.

Two points further complicate this picture: *privatization* and *reduction*. Each thread wraps its code in a `SPriv` command that creates private copies of variables as specified in the `SPar` directive’s `private` clause. Furthermore, if the directive’s `reduction` clause is non-empty, the threads must execute additional commands (denoted by `SBRB`) to perform reduction at the end of the region. We discuss privatization and reduction in Section 3.5.

The For Construct The `for` construct is the most common form of work-sharing in OpenMP, distributing the iterations of a `for` loop across the threads in a team. For example, the program in Figure 3 distributes its loop to a team of 5 threads, each of which executes some portion of the loop in parallel. The program will still print each of 0 to 99 once, but in an arbitrary order⁷ determined by the partition of iterations and the order of parallel execution.

The rule STEP-FOR applies when a thread reaches a `SFor` pragma applied to the loop statement s . The OpenMP standard requires that s be in *canonical loop nest form*⁸, essentially restricting the syntax of the loop so that it is possible

⁷ This loop body is safe to execute in parallel because the iteration variable `i` is made *private*; we explain this in more detail in the following sections.

⁸ Our semantics formalizes requirements about the initializer, the condition, and the increment statements in the loop. The OpenMP specification imposes further re-

to compute the *logical iterations* at runtime by just looking at the initializer, increment, and condition statements of the associated loop. We then choose a partition w that maps thread ID to a list of iterations, so $w(i)$ is the iterations for thread i . The partition w is arbitrary as long as these iterations combine to a permutation of the original iterations. The team must agree on a context `ForCtx` for the loop that includes w , guaranteeing that each thread uses the same partition. This is accomplished using the `sync_ectx(ForCtxidx w)` operation: if there is no `ectx` for its team in $\mathcal{T}$, it is set to be `ForCtxidx w`; otherwise, if there is already some `ectx` set, `sync_ectx` is only defined if `ectx = ForCtxidx w`, essentially enforcing the team to agree on the same w . The thread then executes a modified version s' of the loop, where the iterations have been restricted to its piece $w(i)$ by modifying the initialization and test expressions, followed by a closing barrier. As with `parallel`, any privatization and reduction clauses in the `for` pragma are handled by `SPriv` and `SBRB` respectively.

The Single Construct We also model the simpler work-sharing construct `single`, which says that only one thread in the team executes the associated code. For example, in Figure 4, exactly one thread prints “Before”, and 2 threads print “After”, and “Before” is always printed before any “After”. As with `for`, the threads must agree on a `SinCtx` by `sync_ectx`, but this time its only contents are the id j of the thread that will execute the code. Threads that are not chosen execute no instructions (`SSkip`) and privatize no variables (`[]`) in the region. The `SSingle` construct does not support reduction clauses, since only one thread performs computation in the region.

<pre> 1 #pragma omp parallel num_threads(5) 2 { 3 printf("before\n"); 4 #pragma omp barrier 5 printf("after\n"); 6 }</pre>	<pre> 1 #pragma omp parallel num_threads(2) 2 { 3 int i = omp_get_thread_num(); 4 if (i==0) { bar1(); } 5 else { bar2(); } 6 }</pre>
(a) threads wait at a barrier	(b) Threads in a team should enter barriers in the same order

Fig. 5: OpenMP programs with the `barrier` constructs.

Barriers The `barrier` construct creates an explicit barrier that blocks a thread until all threads in its team reach the barrier. For example, in Figure 5a, all 5

quirements on the loop body: there must be no `break` or `continue` statements, and the iteration variable must not be modified during execution of the loop body, preventing partial or skipped execution of some iterations. Since this is a syntactic constraint, we assume that a parser has already implemented this check when it generates the ClightOMP language.

threads have to reach the barrier before any of them can proceed, so all “before”s will be printed before any “after”.

When thread i takes a STEP-BARRIER, it first identifies its team members:

Definition 2 The team of a thread i in a team tree $\mathcal{T}$ is the set of thread IDs of all the siblings of $\mathcal{T}(i)$ (including i), denoted as `team_tids`($\mathcal{T}, i$).⁹

STEP-BARRIER requires all threads in this team wait at the *same* barrier, and the *idx* of the barrier distinguishes syntactically distinct barriers. Consider the program in Figure 5b: `omp_get_thread_num`¹⁰ returns 0 for the leader and 1 for the other thread, so two threads in a team arrive at different barriers and should be stuck according to the OpenMP specification.¹¹

Our semantics gives uniform treatment to both explicit `barrier` directives and the implicit barriers at the ends of regions (i.e., the barriers in SBRB generated by rules like STEP-PARALLEL), and we index `SBarrier` _{idx, b, s} with not just *idx* but an additional boolean *b* and an optional statement *s* (the directive that generates the implicit barrier, or $\perp$ for an explicit barrier) to handle the subtle differences in their semantics. As shown in the rule STEP-BARRIER, a barrier can only execute when a full team of threads are all at the same barrier (i.e., a barrier with the same indices idx, b, s). Explicit barriers must not be placed inside team-executed regions, which we check via the team’s `ectx`; this requirement comes from the OpenMP specification and serves to rule out, e.g., the case where a barrier in the body of a shared loop is encountered a different number of times by threads executing a different number of iterations of the loop body. Further, if the barrier is the closing operation ($b = \text{true}$) for some construct *s*, it removes related context from $\mathcal{T}$ accordingly with `end_ctx`: for a `for` construct ($s = \text{SFor}$) or a `single` construct ($s = \text{SSingle}$), it removes the `ectx`; for a `parallel` construct ($s = \text{SPar}$), it removes the team (i.e. undo the effects of `spawn_team`).

Finally, all threads in the team simultaneously move to the next statement, and may also freely exchange permissions within the team: for instance, a barrier may be used to separate a code segment in which a thread has exclusive `Writable` access to a variable from one in which `Readable` permission is shared across the whole team.

3.5 Privatization and Reduction

As described in Section 1.1, OpenMP directives can have subtle effects on the semantics of variables within the affected regions. In particular, both `parallel`

⁹ Notice a sibling node of $\mathcal{T}(i)$ is not necessarily active since it may spawn another team, but only the thread of the sibling node is in `team_tids`($\mathcal{T}, i$).

¹⁰ An OpenMP thread number uniquely identifies a thread in a team. For thread i , the function `omp_get_thread_num` returns the index of node $\mathcal{T}(i)$ relative to its siblings. Note that this is different from a thread ID, which uniquely identifies a thread in the thread pool.

¹¹ This program terminates for OpenMP implementations in GCC or Clang, and to the best of our knowledge, there are currently no tools that catch this issue.

and `for` constructs can trigger *privatization* and *reduction*, according to the private clause *pc* and the reduction clauses *rcs* included in their directives. We model the semantics of these clauses as additional helper statements emitted at the start of each region, with arguments derived from both the syntactic clauses and the program's runtime state. The rules are defined in Figure 6.

$$\begin{array}{c}
\text{STEP-PRIV} \\
\frac{\text{priv}_{\vec{x},i}(le, m) = \langle le', m' \rangle}{\text{SPriv } \vec{x} \ s \parallel le \parallel m \mapsto_i s; \text{SPrivEnd } le |_{\vec{x}} \parallel le' \parallel m'} \\
\\
\text{STEP-PRIV-END} \\
\frac{\text{end_priv}_{i,le_o}(le, m) = \langle le', m' \rangle}{\text{SPrivEnd } le_o \parallel le \parallel m \mapsto_i \text{SSkip} \parallel le' \parallel m'} \\
\\
\text{STEP-RED} \\
\frac{r = \langle \vec{rc}, le_o \rangle \quad \vec{t} \triangleq \text{team_tid}(\mathcal{T}, i)}{\text{SRed } r \parallel le \parallel m \xrightarrow{i, \mathcal{P}, \mathcal{T}} \text{SSkip} \parallel le \parallel \text{if } \text{leader}(\mathcal{T}, i) = i \text{ then } \text{red}_{\vec{t}, \vec{rc}, le_o, \mathcal{P}}(m) \text{ else } m}
\end{array}$$

Fig. 6: Semantic rules: `private`, `reduction`. Supporting operations `priv`, `end_priv` and `red` are defined in Figure 13 in the appendix

Privatization By default, variables declared before a parallel region are *shared* among the threads in the region's team: they are treated as memory locations that can be accessed by all threads (and access must be synchronized accordingly). However, sometimes it is useful for each thread in the team to have its own *private* copy of a shared variable. In the example in Figure 7, if `x` were a shared variable, the threads' accesses to it would race; the intended behavior is instead that each thread uses its own copy of `x`, as indicated by the `private` clause in the `parallel` directive. Both `parallel` and `for` constructs can include `private` clauses; the iteration variable of a `for` construct is also implicitly privatized as mentioned in Section 3.4 (we assume that the parser has already added any implicitly privatized variables to the relevant `private` clauses).

In Figure 7, if there were no `private` clause, variable `x` would be shared in the team in the parallel region, and updates to `x` on line 5 would race. With the `private` clause, a private copy of `x` is allocated for each thread in the team, so it is safe to update `x` on line 5, and on line 6 each thread's `x` holds its own OpenMP thread number. After the parallel region, these private copies of `x` are deallocated, and the original `x` is again in scope.

The execution of a region with a `private` clause is prefixed with a statement `SPriv` $\vec{x}$, where $\vec{x}$ is the list of variables to be privatized. The rule STEP-PRIV describes the semantics of privatization. For each private variable $x_k \in \vec{x}$, we allocate a new location l_k in the shared memory m which will hold the value of thread i 's private copy of x_k , and set x_k to refer to l_k in i 's local environment. The initial value at l_k is undefined, as if it were a newly declared variable (with the exception that variables in reduction clauses are implicitly privatized and initialized¹²). We then add an `SPrivEnd` instruction to the end of the region,

¹² OpenMP has rather detailed rules for determining the initial values of reduction variables, which we do not discuss here but are reflected in our semantics.

```

1 int x = 3;
2 #pragma omp parallel private(x) num_threads(2)
3 { // x uninitialized
4     x = omp_get_thread_num();
5     printf("my thread number is %d\n", x);
6 } // x=3

```

Fig. 7: An OpenMP program with a `private` clause, adapted from [13].

with an argument le_o containing the original memory locations of the variables in $\vec{x}$. When we reach the `SPrivEnd` statement, we reverse the process, freeing each location allocated for a private variable and restoring the local environment to the original locations of $\vec{x}$. Note that each thread in a team will separately execute the `SPriv` instruction, allocating its own copies of the variables in $\vec{x}$; after all threads end privatization, each x_k will once again refer to the same memory location in each thread.

```

1 int sum=0;
2 #pragma omp parallel num_threads(2)
3 {
4     #pragma omp for reduction(+:sum)
5     for(int i=1; i≤100; i++) { sum += i; }
6 }
7 printf("sum of the first hundred numbers is %d\n", sum);

```

Fig. 8: An OpenMP program with a `reduction` clause

Reduction A reduction variable is a special kind of private variable for which, instead of returning to the original value after the region ends, we apply some operation to combine all of the private copies into a new value for the shared variable. For example, in Figure 8, the loop iterations are divided between two threads, each summing a part of the 100 numbers. Each thread saves its running sum in an implicitly privatized `sum` variable during the `for` construct. When both threads are at the end of the `for` construct, the reduction clause says that the private copies of `sum` are combined with the original one with the operator `+`, producing the total sum.

We assume that all reduction variables are declared to be private (the parser can add all reduction variables to the `private` clause of the same construct), so they are already processed by the `SPriv` statement for their region. The additional work of reduction occurs at the end of the region: if the region's `reduction` clause is non-empty, then instead of a single barrier, `SBRB` emits a barrier, a reduction statement `SRed`, and another barrier¹³. The first barrier synchronizes all threads in the team, ensuring that they have all finished the

¹³ We do not support the `no_wait` clause that skips these implicit barriers.

region, and transfers permissions for all their reduction variables to the leader. The `SRed` statement is executed only by the leader and performs reduction on all private copies of each reduction variable, folding the specified operation over their values and storing the result in the original variable. (For all non-leader threads, `SRed` is a no-op.) Finally, the last barrier forces all threads to wait until the reduction is complete before proceeding to any remaining code, and it also redistributes the team’s permissions.

3.6 Design Considerations

There are many different ways of translating the OpenMP specification document into operational semantics. Our semantics is aimed to uncover variable and synchronization errors, and so the following considerations are especially important:

Barrier is the only operation that synchronizes running threads. An operational semantics is easiest to reason with when each step is performed by one thread, and concurrency can be expressed by the interleaving of steps of individual threads. Every one of our rules except STEP-BARRIER is local in this sense: each thread may enter a `for` or `single` region, or execute privatization or reduction, independently of all other threads, with coordination accomplished via the asynchronous `sync_ctx` (the first thread to enter a region sets the `ctx`, and the others read it). Only STEP-BARRIER requires that all threads in a team be at the same instruction and moves all threads forward simultaneously. Thus, all synchronization in an execution can be traced specifically to barrier operations (or to thread creation via STEP-PARALLEL).

Operations that can race are not atomic. If two operations can occur in either order, it is important that our semantics observe the resulting nondeterminism, which is hidden if the operations are both performed as part of a single step. For example, we could have combined STEP-END-PRIV and STEP-RED-LEADER into a single end-of-region step, but since both access privatized variables, this would obscure possible races between reduction and ending privatization. By separating them, our semantics identifies the need for a barrier between the two steps¹⁴, and could identify possible races if, e.g., we implemented the `nowait` clause that removes the barrier at the end of a region. Another consequence of this separation is that there is no step that both synchronizes threads and accesses memory, which leads to the next point.

A thread must have permission to perform its memory accesses. Steps that access memory (such as privatization and reduction) only succeed if the thread performing them has the required permissions at the start of the step. This allows the permissions to serve their intended purpose of race detection: the fact

¹⁴ This synchronization is not explicitly mentioned in the OpenMP specification, but its absence leads to races in programs that should be well-defined, and both GCC and Clang place a barrier before reduction.

that permissions are always consistent in an execution state is enough to guarantee the absence of data races. We discuss race-freedom further in Section 4.2.

As a side note, we are also able to support the OpenMP `critical` construct by elaborating it to `acquire` and `release` of a dedicated lock, since lock operations are already supported by the CPM.

4 Properties of the ClightOMP Semantics

The ClightOMP semantics inherit from Clight and the CPM some safety properties that are implicitly proved as part of demonstrating an execution. Moreover, demonstrating an execution also guarantees that the execution is free of data races.

4.1 Invariants and Safety Properties

First, Clight’s semantics enforces memory safety:

Theorem 1 (from CompCert). *If a thread uses the value of a variable, then that variable has a defined value. If a thread step accesses memory, then that memory access is valid.*

So proving that a program (even a sequential one) can execute under our semantics implies that threads never use uninitialized variables, access unallocated or deallocated memory, access arrays out of bounds, etc.

Second, the CPM enforces *permission coherence* before each step: each thread’s permissions must be consistent with each other thread’s, i.e., no two threads can have write access to the same location at the same time.

Theorem 2 (from CPM). *If an execution from the initial state of a program reaches a state $\langle \mathcal{U}, \mathcal{P}, \mathcal{T}, m \rangle$, then the permissions of threads in $\mathcal{P}$ are coherent.*

This is most relevant to our semantics when we perform the `sync` operation: we may redistribute permissions freely within the team at team creation and at barriers, but the new distribution must always be coherent. This is useful for proving race freedom, as we will see in the next section.

As a consequence of these properties, demonstrating an execution of a program under our semantics implies the absence of several categories of common bugs. Variable initialization, memory errors, and data races are responsible for many real-world errors in OpenMP programs [1], so even without proving specific properties of a program (e.g., functional correctness), showing that it executes according to ClightOMP makes it much more likely to be correct.

4.2 Race-Freedom

The CPM’s permissions are intended to guarantee race-freedom of any execution under its semantics, and its authors prove that at the x86-TSO level, successful executions contain no data races [6]. Our semantics enjoys a similar property at the C level, using the fact that every thread only performs memory operations that are allowed by its current permissions:

Theorem 3 (Data race freedom). *If an execution contains two conflicting events e_1 and e_2 on the same memory location, then e_1 and e_2 are synchronized.*

The proof is in Appendix F. Thus, any successfully terminating execution in our semantics has not encountered undefined behavior. This also justifies our use of a sequentially consistent memory model, since OpenMP specifies that race-free programs without weak-memory `atomic` directives (which we do not support) have sequentially consistent behavior.

5 Related Work

C Semantics and Concurrency. Our work builds on the Concurrent Permission Machine [6], which is part of an effort to modularly lift CompCert’s semantics and correctness proof to concurrency. Other approaches to adding concurrency to CompCert include CompCertTSO [18], which directly extends CompCert with the TSO memory model, and CASCompCert [8], which is similar to the CPM but works with memory footprints instead of permissions. CASCompCert’s semantics are simpler than the CPM’s and do not involve as much explicit permission management, but it requires that concurrency primitives be implementable as atomically executed blocks of ordinary C commands, which does not permit the team management involved in OpenMP directives.

Two other major formalizations of C’s semantics are KC [7] and Cerberus [15, 14]. The former does not include concurrency at all, but the latter does, and could also be extended with OpenMP support. While CompCert/the CPM has the advantage of connecting to a verified compiler, Cerberus has an associated model checking tool that is useful for detecting unusual concurrent behaviors.

OpenMP Semantics and Verification. Atzeni and Gopalakrishnan [3] previously gave semantics to OpenMP for the purposes of defining a sound race detector. They also organize the thread hierarchy in a tree-like data structure similar to our team tree, with a different mechanism, *offset-span label*, to describe specific positions within the structure. However, their work is focused entirely on race detection: it elides the semantics of actual source programs, and models program execution as a stream of memory operations and concurrency events (e.g., `parallel` and `barrier` directives). Thus, there are several classes of OpenMP errors that are invisible to their semantics, including malformed control flow/region nesting and reading from uninitialized private variables. In fact, even the `for` construct is not explicitly modeled in their semantics, since its effects are entirely at the level of variables and control flow.

The CIVL concurrency verifier [16] supports verification of OpenMP programs, as well as other concurrency frameworks such as MPI and CUDA. It does so by translating OpenMP directives into concurrency primitives in CIVL-C, a custom C extension with built-in concurrency operations. While CIVL-C has formal semantics and an associated verifier via SMT solvers, the translation of OpenMP to CIVL-C is part of CIVL’s trusted computing base. Our work provides a means to verify this translation by showing that the CIVL-C implementation of OpenMP is consistent with its source-level semantics.

6 Conclusion and Future Work

In this work, we have presented formal semantics for C programs with OpenMP directives. Our semantics extends the concurrent C semantics of the Concurrent Permission Machine with a *team tree* that tracks the states of threads spawned via OpenMP, and carefully accounts for the effects of privatization and reduction on local variables. This gives us a formal definition of the allowed behaviors of OpenMP C programs according to the OpenMP specification, and enables us to identify subtle, undesirable behaviors induced by incorrect annotations; in particular, we have shown that any execution in our semantics is data-race-free. Because most of the OpenMP logic is captured by changes to the team tree, our semantics are also fairly modular and could be reused, with minimal modification, to define OpenMP semantics for other languages (e.g., C++ or Fortran).

We envision two main applications for this semantics: *testing* and *verification*. On the testing side, we aim to develop an *executable* version of our semantics that could serve as a reference interpreter for OpenMP programs. An executable semantics would allow us to test our semantics against actual OpenMP implementations, potentially uncovering bugs in either our semantics or the implementations. It could also serve as a basis for property-based testing (PBT), automatically verifying the correctness properties of OpenMP programs in a sound manner grounded in our semantics. The PBT framework would be useful for detecting race conditions and runtime errors in OpenMP programs.

On the verification side, tools that automatically parallelize sequential programs by instrumentation with OpenMP pragmas [17, 12, 11] lack a semantic preservation guarantee. To aid this, we can prove that a program is correctly instrumented with OpenMP directives by proving that its allowed behaviors under our semantics are a subset of those of the original sequential program (i.e., refinement). We are also interested in using our semantics to verify a *compiler* that implements OpenMP directives with standard concurrency primitives: the CPM already has a (mostly) proved-correct compiler to assembly, so by translating OpenMP directives into CPM spawn and lock operations, we can obtain a verified compiler for C+OpenMP. In the longer run, we note that the CPM was developed as backing for the Verified Software Toolchain (VST) [2], a tool for verifying functional correctness of C programs; our semantics could serve as the basis for an extension of VST to verify OpenMP programs as well, including those with more complex behavior that may not exactly match the behavior of sequential programs (at least locally).

A ClightOMP Syntax

Identifier: x	Integers: $n ::= \mathbb{Z}$	Booleans: $b ::= \text{true} \mid \text{false}$
$rid ::= + \mid * \mid \& \mid \mid \sim$	reduction_identifier	
$ \quad \&\& \mid \mid \max \mid \min$		
$nc ::= n$	num_thread clause	
$pc ::= x^*$	privatization clause	<code>private(x*)</code>
$rc ::= (rid, x^*)$	reduction clause	<code>reduction(rid:x*)</code>
$r ::= (rc^*, le)$	data for reduction	
$s ::= cs$	Clight Stmt	
$ \quad \text{SPar}_n nc pc rc^* s$	parallel construct	<code>#pragma omp parallel</code>
$ \quad \text{SFor}_n pc rc^* s$	for construct	<code>#pragma omp for</code>
$ \quad \text{SSingle}_n pc s$	single construct	<code>#pragma omp single</code>
$ \quad \text{SBarrier}_{n,b,s}$	barrier construct	<code>#pragma omp barrier</code>
$ \quad \text{SPriv} pc s$	start a privatization scope	
$ \quad \text{SPrivEnd} le$	end of a privatization scope	
$ \quad \text{SRed} r$	reduction	

Fig. 9: ClightOMP Syntax. `SPriv`, `SPrivEnd`, `SRed` are generated at runtime, and le stores runtime values.

B Nested Parallel Region

Figure 11 depicts the team tree states in a possible execution of the program from Figure 10. The program starts with a single thread $t1$, just like a sequential C program, and the corresponding initial team tree, as depicted in Figure 11(a). The subscript i of an OpenMP thread state denotes the thread ID, and the superscript indicates the level of nesting in a parallel region, or *init* to specify a freshly initialized node.

When thread $t1$ reaches the first `parallel` construct on line 2, it starts a new team that executes the code associated with this construct (lines 3–9).

As depicted in Figure 11(b), the `parallel` construct on line 2 starts a new parallel region, triggering `spawn_team` that adds a new team of two nodes of thread IDs $t1, t2$ (along with some team context that is not shown here). The new node s_{t1}^2 supersedes the original node s_{t1}^1 for thread $t1$, and starts with its own initial (empty) stack of OpenMP contexts, representing the fact that “ $t1$ as part of the team $\{t1, t2\}$ ” is a different logical entity from “ $t1$ as the

initial thread”. When the team ends at line 9, although $t2$ will terminate and $t1$ will continue to execute, both nodes in the team are removed from the tree by `despawn_team`; $t1$ will return to OpenMP state s_{t1}^1 , its state from before the `parallel` construct (Figure 11(d)).

Nested `parallel` constructs create nested parallel regions. As depicted in Figure 11(c), when $t1$ of the team $\{t1, t2\}$ meets the parallel construct on line 4 in Figure 10, it spawns a new team with the same rules, while $t2$ has not reached line 4 yet. When $t2$ reaches line 4, it likewise spawns a new team $\{t2, t5, t6\}$ for parallel region III for some new threads $t5, t6$, and the execution of this team does not interfere with the other team $\{t1, t3, t4\}$.

```

1 // program starts with one thread t1 (Parallel Region I)
2 #pragma omp parallel num_threads(2) // (Parallel Region II)
3 { // a team of 2 threads {t1, t2} is created
4     #pragma omp parallel num_threads(3) //(Parallel Region III)
5         { // t1 creates a team of {t1, t3, t4}, t2 creates {t2, t5, t6}
6             do_something
7         } // the teams {t1, t3, t4}, {t2, t5, t6} ends here, the primary
8           // thread of each team t1 and t2 resumes to parallel region II
9     } // the team {t1,t2} ends
10 // t1 resumes in the outmost parallel region I

```

Fig. 10: An OpenMP program with nested parallel regions I, II and III, marked with dotted squares.

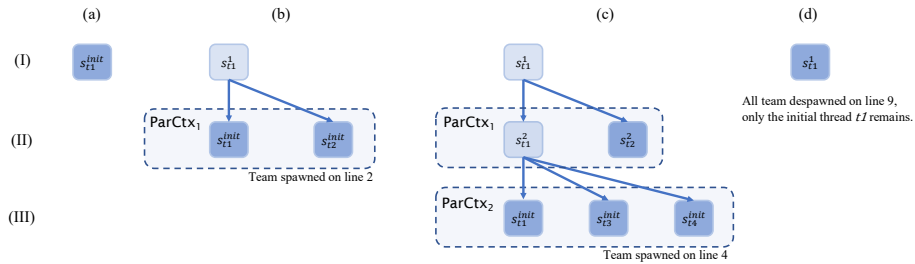


Fig. 11: Team tree $\mathcal{T}$ evolution for an execution of the program in Figure 10. Blue nodes are active; grayed out ones are inactive. (I), (II), and (III) refer to the regions in Figure 10. Each team also has an associated parallel context.

C Team Tree Operations

Node operations of the form $\text{op} \triangleq \langle r, f \rangle$ is a pair of two functions $r, f : \text{Node} \rightarrow \text{Node}$, in which f updates a node, and r finds the node that needs to be updated with respect to the input node: id is the identity and parent finds the parent node. n in the when clause binds to the node before applying op , and op is defined only if the when clause holds.

$$\begin{aligned}
\text{spawn_team}(pctx, \vec{t}) &\triangleq \langle \text{id}, \lambda n, n.\text{tm} ::= \langle pctx, \perp, \text{new_nodes}(\vec{t}) \rangle \text{ when } n.\text{tm} = \perp \rangle \\
\text{despawn_team}(idx) &\triangleq \langle \text{parent}, \lambda n, n.\text{tm} ::= \perp \text{ when } n.\text{ctx} = \perp \wedge \\
&\quad n.\text{pctx} = \text{ParCtx}_{idx} \wedge \forall n' \in n.\text{mates}, n'.\text{tm} = \perp \rangle \\
\text{sync_ctx}(ctx) &\triangleq \langle \text{parent}, \lambda n, n.\text{ctx} ::= ctx \text{ when } n.\text{ctx} = \perp \vee n.\text{ctx} = ctx \rangle \\
\text{pop_ctx}() &\triangleq \langle \text{parent}, \lambda n, n.\text{ctx} ::= \perp \text{ when } n.\text{ctx} \neq \perp \rangle \\
\text{end_ctx}(idx, s) &\triangleq \text{if } s = \text{SFor} \vee s = \text{SSingle} \text{ then } \text{pop_ctx} \text{ else } \text{despawn_team}(idx)
\end{aligned}$$

Tree operations $\text{op}(\mathcal{T}, i)$ on a tree T and a thread ID i , lifted from node operations with the same name. op to the right of $\triangleq$ binds to the node operations:

$$\begin{aligned}
\text{op}(\mathcal{T}, i) &\triangleq \text{op}.1(\mathcal{T}(i)) ::= \text{op}.2(\text{op}.1(\mathcal{T}(i))) \\
&\quad \text{where } \text{op} \in \{\text{spawn_team}(pctx, \vec{t}), \text{despawn_team}(idx), \text{sync_ctx}(ctx), \\
&\quad \quad \text{pop_ctx}(), \text{end_ctx}(idx, s)\} \\
\text{leader}(\mathcal{T}, i) &\triangleq \text{Thread ID of the } leader \text{ of the team of } i \\
\text{team_tids}(\mathcal{T}, i) &\triangleq \text{Thread IDs of all teammates in the team of } i
\end{aligned}$$

Fig. 12: Team tree operations $\text{op}(\mathcal{T}, i)$, $\text{leader}(\mathcal{T}, i)$, $\text{team_tids}(\mathcal{T}, i)$ and the node operations that $\text{op}(\mathcal{T}, i)$ is lifted from.

D Supporting Privatization and Reduction Operations

We first define some notations:

$$\begin{aligned}
\circ_j^{[t_0; t_1; \dots; t_k]} f_j &\triangleq f_{t_k} \circ \dots \circ f_{t_1} \circ f_{t_0} \text{ where } \forall j \in [t_0; t_1; \dots; t_k], f_j : T \rightarrow T \\
d[k_0; \dots; k_n] ::= [v_0; \dots; v_n] &\triangleq d[k_0 ::= v_0] \dots [k_n ::= v_n] \\
f(\vec{t}) &\triangleq \text{map } f \vec{t} \text{ where } f : T \rightarrow T, \vec{t} : \text{List } T
\end{aligned}$$

The privatization operation priv takes the local variable environment and memory right before privatization, allocates private copies of $\vec{x}$ in m , and updates le . The end of privatization operation end_priv frees variable names in le_o , and restores local variable environment for these privatized variables. The reduction operation red gathers contributions in threads $\vec{t}$, adds them to the original variable (by looking up the original local variable environment le_o before privatization).

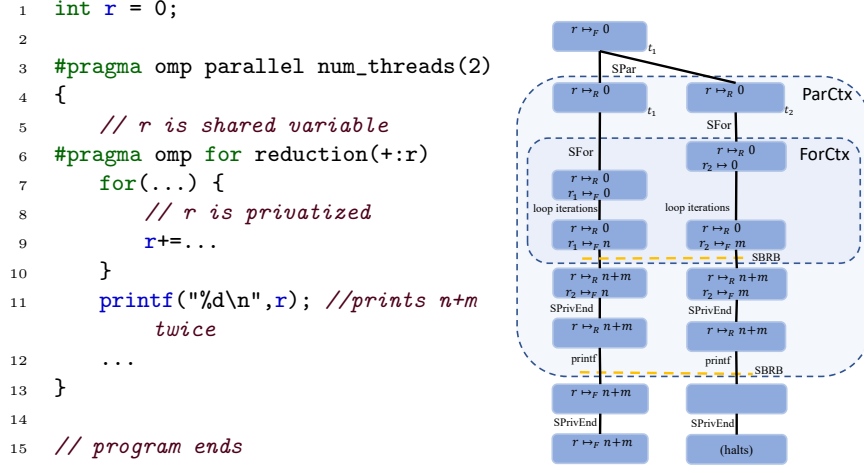
$$\begin{aligned}
\text{priv}_{\vec{x},i}(le, m) &\triangleq \text{let } \langle m', \vec{l}' \rangle := m. \text{alloc}(\text{ctypes}_{le}(\vec{x})) \text{ in} \\
&\quad \langle le[\vec{x} ::= \vec{l}'], m' \rangle \\
\text{end_priv}_{i,le_o}(le, m) &\triangleq \text{foldr } (\lambda x. \\
&\quad \text{let } m' := m. \text{free}(le(x)) \text{ in} \\
&\quad \langle le[x ::= le_o(x)], m' \rangle \\
&\quad) \langle le, m \rangle \text{ dom}(le_o) \\
\text{red_one_var}_{\vec{t},rop,x,v_o,\mathcal{P}}(m) &\triangleq \text{let } \vec{v} := \text{map } (\lambda t. m((\mathcal{P}(t).le)(x))) \text{ in} \\
&\quad \text{let } v' := \text{foldr } (\lambda a b. b. \text{eval_expr}(\text{EBinOp } rop \ b \ a)) \ v_o \ \vec{v} \text{ in} \\
&\quad m[l_o ::= v'] \\
\text{red_one_clause}_{\vec{t},rc,le_o,\mathcal{P}}(m) &\triangleq \text{let } (rop, \vec{x}) := rc \text{ in} \\
&\quad \bigcirc_{\vec{x}}^{\vec{s}} \text{red_one_var}_{\vec{t},rop,x,m(le_o(x)),\mathcal{P}}(m) \\
\text{red}_{\vec{t},\vec{rcs},le_o,\mathcal{P}}(m) &\triangleq \bigcirc_{rc}^{\vec{rcs}} \text{red_one_clause}_{\vec{t},rc,le_o,\mathcal{P}}(m)
\end{aligned}$$

Fig. 13: Supporting functions for privatization and reduction. $\mathcal{P}(t).le$ returns the local variable environment of the thread state $\mathcal{P}(t)$. **alloc**, **free** allocates and frees a chunk of memory; **ctypes** looks up the Clight types of variables; **eval_expr** evaluates a Clight expression to a value. These are defined as in CompCert. **dom** returns the domain of a map as a list.

E An Example ClightOMP Execution

We demonstrate how a program executes in our semantics in Appendix E. We show the program on the left, and each line's corresponding active threads on the right. The program begins with an initial thread t_1 in the thread pool and an initial team tree $\mathcal{T}$. When t_1 executes the parallel pragma on line 3 with the rule STEP-PARALLEL, it forks another thread t_2 and makes a new team $\{t_1, t_2\}$. At this point, the variable name **r** in both threads refers to the same variable, and they can read **r** concurrently. Then t_1 and t_2 each enter the **for** construct individually with the rule STEP-FOR: the associated STEP-PRIV privatizes the reduction variable **r** and initializes it to 0. Each thread is assigned part of the loop iterations (the assignment is set in the ForCtx when the first thread reaches the **for** pragma), and running these iterations updates **r** to be n in t_1 and m in t_2 by the end of the loop. The first thread reaching line 10 is then paused at the initial barrier of SBRB, until the other thread reaches line 10 and triggers STEP-BARRIER, and t_2 gives its permission on the original variable **r** to t_1 . Then each thread executes SRed, which in t_2 does nothing and in t_1 combines the threads' contributions n, m with the original value 0, setting the original version of **r** to $0 + n + m$. After another barrier at the end of SBRB, the ForCtx's lifetime ends and the threads each execute SPrivEnd, deallocating their private copies of **r** and returning the name **r** to refer to the original version. Therefore, after

they exit the `for` construct, both threads will execute line 11 and print the same value $n + m$. Eventually they reach the end of the `parallel` construct on line 13, where the `SBRB` ends the team, removes the team's contexts and nodes from the team tree, and transfers all of t_2 's permissions to r to t_1 (so now t_1 has full permissions). Finally, the program ends with just t_1 in the thread pool and a node for t_1 in $\mathcal{T}$, and we conclude that the program terminates successfully.



F Data Race Freedom

We prove that any ClightOMP execution is race-free. We begin by defining the memory and synchronization events produced by a ClightOMP execution.

$$\begin{aligned}
 (\text{CompCert}) \text{ MemEv} &\triangleq \text{Alloc}_i \ l \mid \text{Free}_i \ l \mid \text{Read}_i \ l \mid \text{Write}_i \ l \\
 \text{SyncEv} &\triangleq \text{Par}_i \ \vec{t} \mid \text{Bar} \ \vec{t} \\
 \text{Event} &\triangleq \text{MemEv} \mid \text{SyncEv}
 \end{aligned}$$

STEP-CLIGHT in thread i	list of MemEv_i
privatization/reduction steps in thread i	list of MemEv_i
STEP-PARALLEL in thread i spawning $\vec{t}$	$[\text{Par}_i \ \vec{t}]$
STEP-BARRIER synchronizing $\vec{t}$	$[\text{Bar} \ \vec{t}]$
other steps	$\square$

Fig. 14: ClightOMP memory events emitted by a step.

A ClightOMP step that accesses memory emits a list of CompCert memory events; a ClightOMP step that performs synchronization emits a synchronization event. Figure 14 shows the events emitted by each step. Per-thread Clight steps and STEP-PRIV/STEP-PRIV-END/STEP-RED-LEADER/STEP-RED-NOT-LEADER emit a sequence of memory events corresponding to their memory operations, each indexed by the thread number i that runs the step and the accessed memory location l . Specifically, STEP-PRIV emits an Alloc event for each memory allocation and STEP-PRIV-END emits corresponding Free events; STEP-RED-LEADER emits Read events for reading the private copies of the reduction variable, followed by a Write event for updating the original copy. STEP-PARALLEL emits an parallel event $\text{Par}_i \vec{t}$ that synchronizes thread i with the spawned threads $\vec{t}$. STEP-BARRIER emits a barrier event that synchronizes among the team $\vec{t}$. We define a labeled version of our semantics $\langle i \cdot \mathcal{U}, \mathcal{P}, \mathcal{T}, m \rangle \xrightarrow{\vec{e}} \langle \mathcal{U}, \mathcal{P}', \mathcal{T}', m' \rangle$ where each step is labeled with the events it emits; then the event trace of an execution is the concatenation of all events emitted at each step.

As is standard, we define a data race as a pair of conflicting memory events (i.e., events to the same location that are not both Reads) that are not ordered via a happens-before relation [9]. First, we need to define the happens-before relation:

Definition 3 (happens-before) Consider a list $\vec{e}$ of events and two events $e_i, e_j \in \vec{e}$ where $i < j$ are indexes to $\vec{e}$. Then e_i happens before e_j , written $e_i <_{\text{hb}} e_j$, if:

- e_i and e_j are from the same thread, or
- e_i is $\text{Par } \vec{t}$ and the thread of e_j is in $\vec{t}$, or
- one of e_i and e_j is $\text{Bar } \vec{t}$ and the thread of the other is in $\vec{t}$, or
- there is another event e_k such that $e_i <_{\text{hb}} e_k$ and $e_k <_{\text{hb}} e_j$.

We observe that if $e_i <_{\text{hb}} e_j$, then e_i also happens before any following events by e_j 's thread; in fact, when two threads synchronize, all previous events by the first thread happen before all following events of the second thread. This makes it useful to talk about an event synchronizing with a thread, rather than simply with another event.

Definition 4 (synchronizing with a thread) An event e_i synchronizes with a thread t in a trace $\vec{e}$ if for any event e_t by t , if we extend $\vec{e}$ with a new trace $\vec{e}'$ including e_t , then $e_i <_{\text{hb}} e_t$ in $\vec{e} ++ \vec{e}'$.

We can now begin to prove race-freedom. Intuitively, we can trace a permission throughout an execution: Writable permission flows forward to *all* threads that hold at least Readable permission for the rest of the execution, and Readable permission flows forward to *some* thread that holds at least Readable permission for the rest of the execution. We now formalize and prove this intuition in terms of event synchronization.

Lemma 1 (Read synchronization). *Suppose we have an execution ending in a state st whose trace contains an event $e = \text{Read}_i l$, and l is not deallocated in st , i.e., there is some thread that has some permission to l in st . Then there is some thread j such that 1) j has at least Readable permission to l in st , and 2) e synchronizes with j in the trace.*

Proof. By induction on the execution. In the base case, the execution consists of a single step $st_0 \xrightarrow{\vec{e}} st$ where $\text{Read}_i l \in \vec{e}$. A thread only performs operations it has permissions to, and steps that perform Reads do not change permissions, so thread i still has Readable permission to l in st and $\text{Read}_i l$ synchronizes with i .

In the inductive case, we have an execution ending in a state st with a thread j as described, and we consider a next step $st \xrightarrow{e'} st'$. If j still has Readable permission to l in st' , then we are finished. If it does not, then e' must be emitted from a synchronization operation that changed j 's permissions. Whether this operation was a Par or Bar, it synchronized with a set of threads $\vec{t}$ whose new permissions in st' sum to the same result as the sum of the source permissions in st , which included j 's Readable permission to l . Thus, there must be at least one thread $k \in \vec{t}$ that has at least Readable permission in st' , and since e synchronized with j and e' synchronized j with k , e now synchronizes with k in the extended trace.

Lemma 2 (Write synchronization). *Suppose we have an execution ending in a state st whose trace contains an event $e = \text{Write}_i l$ (or Alloc or Free). Then for all threads j that have at least Readable permission to l in st , e synchronizes with j in the trace.*

Proof. By induction on the execution. In the base case, the execution consists of a single step $st_0 \xrightarrow{\vec{e}} st$ where $e \in \vec{e}$. If e is a Write or Free then thread i must have had at least Writable permissions to l in st_0 , so by coherence no other thread had Readable permissions to l ; if e is Alloc then its location must have not been allocated in st_0 , so i is the only thread with permissions to l in st . In either case, i is the only thread in st with at Readable permission to l , and e synchronizes with i since they are in the same thread.

In the inductive case, we have an execution ending in a state st with threads synchronized with e as described, and we consider a next step $st \xrightarrow{e'} st'$ and a thread j with at least Readable permission to l in st' . If j had Readable permission to l in st , then we are finished. If it did not, then e' must be a synchronization operation that changed j 's permissions. Whether this operation was a Par or Bar, it synchronized a set of threads $\vec{t}$ whose old permissions in st summed to a result that included j 's new Readable permission to l . Thus, there must be at least one thread $k \in \vec{t}$ that had at least Readable permission in st , and since e synchronized with k and e' synchronized k with j , e now synchronizes with j in the extended trace.

These dual lemmas show that read or write/alloc/free events are always synchronized in some way with threads that hold Readable permissions after them. We can use this to prove data race freedom:

Theorem 4 (Data race freedom). *If an execution's trace contains two conflicting events e_1 and e_2 , then $e_1 <_{\text{hb}} e_2$.*

Proof. By induction, it suffices to consider the case where e_2 is produced by the last step of the execution, $st \rightarrow st'$, and the execution up through st is race-free. Let t_2 be the thread of e_2 . Since e_1 and e_2 conflict, they are on the same location l and either e_1 or e_2 is a write/alloc/free. In the case where e_1 is a write, t_2 has at least **Readable** permission to l (since e_2 must be at least a read), and so by lemma 2 e_1 synchronizes with t_2 and thus $e_1 <_{\text{hb}} e_2$. In the case where e_1 is a read, l is still allocated in st (since we perform e_2 on it¹⁵), so by Lemma 1 there must be some thread j in st with at least **Readable** permission to l such that e_1 synchronizes with j . But we know that e_2 must be either a **Write** or a **Free**, and in either case t_2 must have at least **Writable** permission to l . By coherence, this means that no other thread has **Readable** permission to l , so the thread j must be exactly t_2 ; once again, e_1 synchronizes with t_2 and thus $e_1 <_{\text{hb}} e_2$.

Thus, any execution in our semantics is necessarily race-free.

¹⁵ Note that Clight never reallocates freed locations, so we do not need to worry about the case where e_2 Allocs l after e_1 reads l .

References

1. Ahmmed, J., Mahmud, Q.I., Shim, J., Li, L., Jannesari, A., Cohen, M.B.: Differential testing for sequential to parallel transformations. In: Proceedings of the 9th International Workshop on Software Correctness for HPC Applications. Correctness '25 (2025)
2. Appel, A.W., Dockins, R., Hobor, A., Beringer, L., Dodds, J., Stewart, G., Blazy, S., Leroy, X.: Program Logics for Certified Compilers. Cambridge University Press (2014), <http://www.cambridge.org/de/academic/subjects/computer-science/programming-languages-and-applied-logic/program-logics-certified-compilers?format=HB>
3. Atzeni, S., Gopalakrishnan, G.: An operational semantic basis for building an OpenMP data race checker. In: 2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). pp. 395–404 (2018). <https://doi.org/10.1109/IPDPSW.2018.00074>
4. Besson, F., Blazy, S., Wilke, P.: A concrete memory model for compcert. In: International Conference on Interactive Theorem Proving. pp. 67–83. Springer (2015)
5. Board, O.A.R.: OpenMP Application Programming Interface. <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf> (2024)
6. Cuellar, S., Giannarakis, N., Madiot, J.M., Mansky, W., Beringer, L., Cao, Q., Appel, A.: Compiler correctness for concurrency: from concurrent separation logic to shared-memory assembly language. Tech. rep., Princeton University (2020)
7. Ellison, C., Rosu, G.: An Executable Formal Semantics of C with Applications. In: Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. pp. 533–544. POPL '12, ACM, New York, NY, USA (2012). <https://doi.org/10.1145/2103656.2103719>, <http://doi.acm.org/10.1145/2103656.2103719>
8. Jiang, H., Liang, H., Xiao, S., Zha, J., Feng, X.: Towards certified separate compilation for concurrent programs. In: Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation. p. 111–125. PLDI 2019, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3314221.3314595>, <https://doi.org/10.1145/3314221.3314595>
9. Lamport, L.: Time, clocks, and the ordering of events in a distributed system. Commun. ACM **21**(7), 558–565 (Jul 1978). <https://doi.org/10.1145/359545.359563>, <https://doi.org/10.1145/359545.359563>
10. Leroy, X.: Formal verification of a realistic compiler. Communications of the ACM **52**(7), 107–115 (Jul 2009). <https://doi.org/10/c9sb7q>, <http://doi.acm.org/10.1145/1538788.1538814>
11. Mahmud, Q.I., TehraniJamsaz, A., Ahmed, N.K., Willke, T.L., Jannesari, A.: Contraph: Contrastive learning for parallelization and performance optimization. In: Proceedings of the 39th ACM International Conference on Supercomputing. pp. 596–610 (2025)
12. Mahmud, Q.I., TehraniJamsaz, A., Phan, H.D., Chen, L., Capotă, M., Willke, T.L., Ahmed, N.K., Jannesari, A.: Autoparllm: Gnn-guided context generation for zero-shot code parallelization using llms. In: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). pp. 11821–11841 (2025)
13. Mattson, T.: A “hands-on” introduction to openMP*. https://www.openmp.org/wp-content/uploads/Intro_To_OpenMP_Mattson.pdf (2013), located at <https://www.openmp.org/resources/tutorials-articles/>

14. Memarian, K., Gomes, V.B.F., Davis, B., Kell, S., Richardson, A., Watson, R.N.M., Sewell, P.: Exploring C Semantics and Pointer Provenance. *Proc. ACM Program. Lang.* **3**(POPL), 67:1–67:32 (Jan 2019). <https://doi.org/10.1145/3290380>, <http://doi.acm.org/10.1145/3290380>
15. Memarian, K., Matthiesen, J., Lingard, J., Nienhuis, K., Chisnall, D., Watson, R.N.M., Sewell, P.: Into the Depths of C: Elaborating the de Facto Standards. *SIGPLAN Not.* **51**(6), 1–15 (Jun 2016). <https://doi.org/10.1145/2980983.2908081>, <https://doi.org/10.1145/2980983.2908081>
16. Siegel, S.F., Zheng, M., Luo, Z., Zirkel, T.K., Marianiello, A.V., Edenhofner, J.G., Dwyer, M.B., Rogers, M.S.: Civi: the concurrency intermediate verification language. In: *SC '15: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. pp. 1–12 (2015). <https://doi.org/10.1145/2807591.2807635>
17. Tehrani, A., Bhattacharjee, A., Chen, L., Ahmed, N.K., Yazdanbakhsh, A., Janesari, A.: Coderosetta: Pushing the boundaries of unsupervised code translation for parallel programming. *Advances in Neural Information Processing Systems* **37**, 100965–100999 (2024)
18. Ševčík, J., Vafeiadis, V., Zappa Nardelli, F., Jagannathan, S., Sewell, P.: CompCertTSO: A Verified Compiler for Relaxed-Memory Concurrency. *J. ACM* **60**(3), 22:1–22:50 (Jun 2013). <https://doi.org/10.1145/2487241.2487248>, <http://doi.acm.org/10.1145/2487241.2487248>